

AUDITORÍA CARTERA PRO RISK LAB

Auditoría Técnica Integral y Seguridad de Software:

Informe Técnico Unificado de Calidad y Vulnerabilidades

Autores:

Xiomara Ocampo Hurtado

Carlos Augusto Aranzazu

María Luisa Londoño

Asignatura:

Tendencias en Ingeniería de Software

Docente:

Carlos Andrés Jaramillo

Universidad Alexander von Humboldt
Facultad de Ingeniería y Ciencias Básicas

Ingeniería de Software

Armenia, Quindío

2026

Índice

1. Introducción y Alcance	3
2. Metodología y Herramientas Utilizadas	3
3. Métricas Globales de Calidad y Seguridad	3
4. Hallazgos Principales de Seguridad (OWASP Top 10)	4
4.1. SEC-001: Inyección SQL en Autenticación y Búsquedas	4
4.2. SEC-002: Ejecución Remota de Comandos (RCE)	4
4.3. SEC-003: Credenciales Expuestas en Código Fuente	4
5. Análisis de Riesgos e Impacto (Matriz)	4
6. Anexo de Evidencias Técnicas (Sección Requerida)	4
6.1. Evidencia SAST (SonarQube)	5
6.2. Evidencia DAST (OWASP ZAP)	5
6.3. Evidencia de Prueba Exploratoria	5
7. Recomendaciones y Plan de Remediación	5

Índice de cuadros

1.	Métricas Consolidadas del Sistema	3
2.	Matriz de Riesgos Identificados	5

1. Introducción y Alcance

El presente documento constituye el Informe Técnico Integral de la auditoría realizada sobre la plataforma **CarteraPro Risk Lab**. El objetivo principal de este análisis independiente es evaluar la calidad interna del código, identificar deuda técnica acumulada, detectar vulnerabilidades de seguridad (tanto estáticas como dinámicas) y analizar la superficie de riesgo introducida por dependencias de terceros.

El alcance del análisis incluye el repositorio completo de la aplicación, evaluando el Frontend desarrollado en *React (Vite)* y el Backend implementado sobre *Node.js / Express* con almacenamiento local en *SQLite*.

2. Metodología y Herramientas Utilizadas

Para garantizar la rigurosidad técnica del proceso, se aplicó un enfoque híbrido compuesto por:

- **Análisis Estático de Seguridad (SAST):** Implementado mediante *SonarQube* para la detección de malas prácticas, code smells y hotspots de seguridad.
- **Análisis Dinámico de Seguridad (DAST):** Pruebas automatizadas de caja negra usando *OWASP ZAP* sobre los endpoints expuestos y la interfaz de usuario.
- **Análisis de Composición de Software (SCA):** Auditoría de empaquetado de dependencias utilizando *NPM Audit* para identificar vulnerabilidades conocidas (CVEs).
- **Pruebas Exploratorias Manuales:** Code-review enfocado en lógica de negocio y bypass de controles criptográficos.

3. Métricas Globales de Calidad y Seguridad

A continuación, se consolidan los indicadores cuantitativos obtenidos de las herramientas automatizadas:

Cuadro 1: Métricas Consolidadas del Sistema	
Métrica de Calidad / Seguridad	Valor Detectado
Líneas de Código Analizadas (LOC)	2,169
Code Smells Totales (SonarQube)	534 (50 Blocker, 484 Critical)
Complejidad Ciclomática Total	291
Cobertura de Pruebas Automatizadas	0 %
Duplicación de Código	0.9 %
Alertas DAST de Seguridad (OWASP ZAP)	11
Dependencias con CVEs Críticos	8

4. Hallazgos Principales de Seguridad (OWASP Top 10)

4.1. SEC-001: Inyección SQL en Autenticación y Búsquedas

Severidad: Crítica

Ubicación: backend/src/controllers/authController.js

Descripción: El endpoint de inicio de sesión concatena directamente los parámetros de entrada del usuario dentro de la consulta SQL sin usar sentencias preparadas ni parametrización. Esto permite a un atacante evadir la autenticación por completo.

```
1 // C digo Vulnerable Detectado
2 const query = 'SELECT * FROM users WHERE email = '${email}' AND
  password = '${password}';
3 db.query(query);
```

Listing 1: Vulnerabilidad de concatenación directa en autenticación

4.2. SEC-002: Ejecución Remota de Comandos (RCE)

Severidad: Crítica

Ubicación: backend/src/controllers/adminController.js

Descripción: El sistema expone un endpoint administrativo que ejecuta comandos del sistema operativo directamente utilizando la función `eval()` o `exec()` pasando un parámetro controlado por el usuario de forma directa.

4.3. SEC-003: Credenciales Expuestas en Código Fuente

Severidad: Alta

Ubicación: backend/src/db/database.js

Descripción: Se identificaron llaves secretas criptográficas y contraseñas de bases de datos quemadas directamente en el código (*hardcoded*), facilitando el compromiso total en caso de acceso no autorizado al repositorio.

5. Análisis de Riesgos e Impacto (Matriz)

6. Anexo de Evidencias Técnicas (Sección Requerida)

Nota para el equipo: En esta sección se deben incluir de manera obligatoria las capturas de pantalla de los dashboards locales generados al ejecutar el software en los pasos posteriores.

Cuadro 2: Matriz de Riesgos Identificados

ID	Categoría	Impacto	Probabilidad	Consecuencia en Negocio
R-01	Seguridad	Crítico	Alta	Exfiltración completa de la base de datos financiera de clientes.
R-02	Operacional	Alto	Alta	Caída del servidor por inyección de comandos destructivos.
R-03	Calidad	Medio	Muy Alta	Incremento exponencial de costos en mantenimiento por falta de pruebas.

6.1. Evidencia SAST (SonarQube)

[Insertar Captura del Dashboard de SonarQube mostrando los 534 code smells y el Quality Gate fallido].

6.2. Evidencia DAST (OWASP ZAP)

[Insertar Captura del Reporte HTML de OWASP ZAP con las alertas críticas de inyección].

6.3. Evidencia de Prueba Exploratoria

[Insertar Captura de la consola ejecutando un comando de bypass exitoso].

7. Recomendaciones y Plan de Remediación

1. **Corto Plazo (Inmediato):** Migrar todas las consultas SQL a mapeadores relacionales (ORM) o usar *sentencias preparadas*. Remover llaves duras y moverlas al archivo `.env`.
2. **Mediano Plazo:** Sanitizar entradas usando librerías especializadas y restringir rutas administrativas con *Middlewares* de JWT firmados de forma segura.
3. **Largo Plazo:** Configurar una canalización de CI/CD que bloquee despliegues si la cobertura de pruebas de SonarQube es menor al 80 %.